# Sous-système de travaux par lots BATCH

Guide de l’utilisateur

BATCH-UG-0101

BATCH version 2.2.6 *** 21-AOÛ-2026

# Table des matières

**Préface** **3**
Public visé . . . 3
Structure du document . . . 4
Conventions . . . 5

**Chapitre 1** **6**

**Introduction** **7**
1.1 Vue d'ensemble . . . 7
1.2 Composants . . . 8
1.3 Comparaison avec les ordonnanceurs périodiques . . . 9
1.4 États d'un travail . . . 10
1.5 Vie d'un travail . . . 11

**Chapitre 2** **12**

**Installation** **13**
2.1 Prérequis . . . 13
2.2 Construire et installer . . . 14
2.3 Ce qui est installé . . . 15
2.4 La bibliothèque d'aide . . . 16
2.5 Lancer le contrôleur avec le système . . . 17
2.6 Vérifier l'installation . . . 18
2.7 Retrait . . . 19

**Chapitre 3** **20**

**Configuration** **21**
3.1 Qualificatifs du contrôleur de travaux . . . 21
3.2 Modifier le service . . . 22
3.3 Réglages fixés à la construction . . . 23
3.4 Où l'utilitaire regarde . . . 24
3.5 Ce qu'un travail hérite . . . 25

**Chapitre 4** **26**

**Travailler avec les travaux par lots** **27**
4.1 Soumettre un travail . . . 27
4.2 Transmettre des données au travail . . . 28
4.3 La sortie d'un travail . . . 29
4.4 Régler le moment de l'exécution . . . 30
4.5 Afficher l'état des travaux . . . 31
4.6 Supprimer un travail . . . 32

**Chapitre 5** **33**

**Dictionnaire des commandes** **34**
BATCH SUBMIT . . . 35
BATCH SHOW . . . 36
BATCH DELETE . . . 37
BATCH RELEASE . . . 38
BATCH SET ENTRY . . . 39
BATCH PURGE . . . 40
BATCH SYNC . . . 41
BATCH QUEUE . . . 42
BATCH ANALYZE . . . 43
BATCHD . . . 44

**Chapitre 6** **45**

**Messages** **46**
6.1 Forme d'une valeur de condition . . . 46
6.2 Forme d'un message . . . 47
6.3 Références des messages . . . 48

**Chapitre 7** **50**

**Dépannage** **51**
7.1 Le contrôleur est injoignable . . . 52
7.2 Le contrôleur ne démarre pas . . . 53
7.3 Un travail ne démarre pas . . . 54
7.4 Un travail échoue . . . 55
7.5 La file est vide après un redémarrage . . . 56
7.6 Le répertoire des journaux grossit . . . 57
7.7 Une commande est refusée . . . 58
7.8 Le contrôleur répond lentement . . . 59
7.9 Rassembler de quoi faire un rapport . . . 60

**Annexe A** **61**

**Fichiers** **62**
Variables d'environnement . . . 63

**Annexe B** **64**

**Formes de spécification du temps** **65**
B.1 Temps relatif . . . 65
B.2 Temps absolu . . . 66
B.3 Mots-clés . . . 67

# Préface

## Public visé

Ce manuel s'adresse à l'utilisateur qui soumet des scripts d'interpréteur pour exécution sans surveillance, et à l'administrateur qui installe et conduit le contrôleur de travaux. La familiarité avec un interpréteur POSIX est supposée ; rien d'autre ne l'est. Aucune connaissance préalable d'OpenVMS n'est requise, encore que le lecteur qui connaît les commandes `SUBMIT` et `SHOW QUEUE` d'OpenVMS reconnaîtra la terminologie comme la syntaxe.

Le programmeur appelé à modifier le sous-système veut plutôt le *Guide du programmeur*. Le présent manuel ne dit rien de la manière dont le logiciel est construit.

## Structure du document

| Partie | Contenu |
|---|---|
| Chapitre 1 | Présente le sous-système, ses composants et la vie d'un travail |
| Chapitre 2 | Installer le sous-système et lancer le contrôleur avec le système |
| Chapitre 3 | Configurer le contrôleur et ce qu'un travail hérite de lui |
| Chapitre 4 | Comment accomplir les opérations courantes |
| Chapitre 5 | Le dictionnaire des commandes, une entrée par commande |
| Chapitre 6 | La forme d'un message, et chaque message avec son sens et la conduite à tenir |
| Chapitre 7 | Dépannage : un symptôme, où regarder, que faire |
| Annexe A | Les fichiers créés et employés à l'exécution |
| Annexe B | Les formes de spécification du temps |

Le lecteur qui installe pour la première fois veut le chapitre 2, puis le 3. Celui qui a le sous-système en marche et veut s'en servir veut le chapitre 4. Celui qui tient un message veut le chapitre 6 ; celui qui tient un symptôme, le chapitre 7.

# Conventions

| Convention | Sens |
|---|---|
| `$` | L'invite de l'interpréteur. Ce qui suit est saisi par l'utilisateur. |
| *italiques* | Un terme que l'on définit, ou une variable à remplacer par une valeur. |
| `MAJUSCULES` | Un verbe de commande, un qualificatif, ou la mnémonique d'une valeur de condition. |
| `[ ]` | Éléments de syntaxe facultatifs. Ne tapez pas les crochets. |
| `{ }` | Un choix obligé parmi les éléments enfermés. Ne tapez pas les accolades. |
| `,...` | L'élément précédent peut être répété. |

Commandes et qualificatifs sont montrés en majuscules pour la lisibilité, mais peuvent être saisis dans n'importe quelle casse.

**Note**

Les sorties reproduites dans ce manuel le sont exactement telles que le logiciel les produit.

**Note**

Le sous-système tient le temps en **UTC** et le montre à l'horloge de qui le lit. Toute heure qu'affiche la commande `SHOW`, et toute heure remise au qualificatif `/AFTER`, est dans le fuseau local ; le journal de la file garde l'UTC, ce qui permet à une file de survivre à un changement de fuseau du nœud. La conversion est faite par les services STARLET `UTIL$BINTIM`, `UTIL$ASCTIM`, `UTIL$GETTIM` et `UTIL$ADD_TIMES`.

# Chapitre 1

# Introduction

## 1.1 Vue d’ensemble

BATCH exécute des scripts d’interpréteur en travaux par lots : un script est placé dans une file d’attente et exécuté plus tard, sans terminal, sous la conduite d’un contrôleur de travaux résident. C’est l’équivalent fonctionnel de la file de travaux par lots d’OpenVMS.

Soumettre un script n’est pas l’exécuter. La soumission attribue un numéro d’entrée dans la file et rend la main aussitôt ; le processus est créé plus tard, si et quand le contrôleur en décide ainsi. C’est à l’entrée, non au processus, que l’on se tient — pour l’interroger, la libérer ou la supprimer.

## 1.2 Composants

| Composant | Rôle |
|---|---|
| `batch` | L’utilitaire de ligne de commande. Soumet, affiche, supprime, libère. |
| `batchd` | Le contrôleur de travaux. Possède la file, ordonnance et crée les processus. |
| `batch.hlb` | La bibliothèque d’aide, en trois langues. |

L’utilitaire ne garde aucun état d’un appel à l’autre : tout ce qui survit à une commande est dans le contrôleur et dans deux fichiers, décrits à l’annexe A.

## 1.3 Comparaison avec les ordonnanceurs périodiques

`cron` répond à la question « quand » : il lance une commande à une heure dite, et ne se soucie ni de ce qui tourne déjà ni de ce qui s'est passé la fois précédente. BATCH répond à la question « combien à la fois » : il tient une file, en sort ce qui peut s'exécuter dans la limite qui lui a été fixée, et garde ce qui a échoué pour qu'on puisse le regarder.

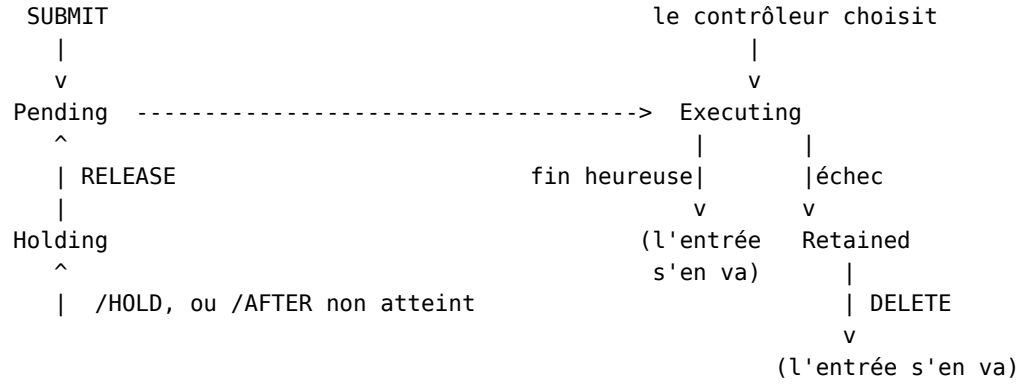
Les deux se complètent : un travail soumis par `cron` à BATCH s'exécutera à l'heure dite *et* dans la limite du parc.

## 1.4 États d’un travail

Un travail se trouve dans exactement un des états suivants.

| État | Sens |
|---|---|
| Pending | Peut s’exécuter et attend une place |
| Holding | Retenu par `/HOLD`, ou par une heure `/AFTER` non atteinte |
| Executing | Un processus lui a été créé et s’exécute |
| Aborting | Un signal de fin a été envoyé et il n’a pas fini |
| Retained | Il a fini et son entrée a été conservée pour examen |

## 1.5 Vie d’un travail




Un travail qui réussit quitte la file ; un travail qui échoue y demeure avec son journal. C’est ce qui rend `SHOW` utile sans qualificatif : ce qui reste est ce qui s’exécute et ce qui a mal tourné.

Cette règle se règle par `/RETAIN` : `ALWAYS` conserve l’entrée quoi qu’il arrive, `NEVER` ne la conserve jamais, `ERROR` — le défaut — la conserve en cas d’échec seulement.

# Chapitre 2

# Installation

## 2.1 Prérequis

| Élément | Exigence |
|---|---|
| Système | Linux muni d’une bibliothèque C conforme POSIX |
| Compilateur | GCC ou Clang acceptant gnu11 |
| CMake | 3.13 ou plus récent |
| UTILITY$ROUTINES | Le paquet StarLet, installé |
| HELP | Le paquet HELP, installé |

Les deux paquets sont exigés en tant que paquets CMake : aucune copie n’en est portée dans cet arbre. UTILITY$ROUTINES est liée en statique, pour la raison exposée au chapitre 7 du *Guide du programmeur*.

## 2.2 Construire et installer

```
$ cmake -S . -B build
$ cmake --build build
$ ctest --test-dir build
# cmake --install build
```

L’installation active et démarre le contrôleur là où elle le peut ; une installation en zone de préparation — faite avec `DESTDIR` — laisse l’un et l’autre à qui installera le paquet.

## 2.3 Ce qui est installé

| Chemin | Contenu |
|---|---|
| `<préfixe>/bin/batch` | L'utilitaire |
| `<préfixe>/sbin/batchd` | Le contrôleur |
| `<préfixe>/share/help/batch.hlb` | L'aide, en anglais |
| `<préfixe>/share/help/ru/batch_rus.hlb` | L'aide, en russe |
| `<préfixe>/share/help/fr/batch_fra.hlb` | L'aide, en français |
| `<préfixe>/share/doc/BATCH/` | Les manuels |
| `/lib/systemd/system/batchd.service` | L'unité systemd, sur un système démarré par systemd |
| `/etc/init.d/batchd` | Le script de démarrage System V, sur un système sans systemd |
| `/var/spool/batch/` | La racine du sous-système |

## 2.4 La bibliothèque d’aide

L’aide s’obtient par la commande du sous-système ou par l’utilitaire général :

```
$ batch help submit /after
$ vhelp batch submit
```

La facilité HELP choisit la langue que demande la locale : `share/help` pour l’anglais, `share/help/ru` et `share/help/fr` pour les traductions.

Après une installation, le catalogue partagé est à reconstruire :

```
# vhelp /SYSCACHE
```

> L’image du navigateur HELP s’appelle `vhelp`, et son bibliothécaire `vhelptran` : `help` est une commande interne de bash et de ses semblables, et une commande interne est cherchée avant le chemin, si bien qu’aucun ordre du `PATH` ne mènerait à une image de ce nom.

## 2.5 Lancer le contrôleur avec le système

L'installation **active et lance** le contrôleur ; `BATCH_M_AUTOSTART=OFF` laisse ce soin à l'administrateur. Un contrôleur qui tourne déjà n'est pas remplacé : après une mise à jour, relancez-le.

Sous systemd :

```
# systemctl enable --now batchd
# systemctl restart batchd          # après une mise à jour
# systemctl status batchd
```

Les diagnostics du contrôleur vont dans le journal du système lorsqu'il tourne en service :

```
# journalctl -u batchd -f
```

Sans systemd, le script `/etc/init.d/batchd` fait de même. L'installation lui donne ses liens par `update-rc.d` et lance le contrôleur par lui :

```
# update-rc.d batchd defaults
# service batchd start
# service batchd restart            # après une mise à jour
# service batchd status
```

Le script est installé dans `BATCH_K_INITDIR` (`/etc/init.d`) lorsque `BATCH_M_SYSVINIT` vaut `ON`, ce qui est le cas par défaut sur un système que systemd n'a pas démarré.

Lancé par ce script ou à la main, il écrit dans `<racine>/batchd.log` et se détache ; le terminal qu'il quitte est averti de ce qu'il est advenu et du fichier où lire la suite.

## 2.6 Vérifier l’installation

```
$ echo 'echo bonjour ; sleep 2' > /tmp/essai.sh
$ batch submit /tmp/essai.sh /NAME=essai /KEEP
%BATCH-S-QUEUED, job essai queued on SYS$BATCH as entry 1
$ batch show /all
$ cat essai_1.log
```

## 2.7 Retrait

```
# cmake --build build --target uninstall
```

Le retrait arrête et désactive le service avant qu'aucun fichier ne s'en aille : une unité retirée alors que le service est encore activé laisse systemd se plaindre à chaque démarrage d'une unité qu'il ne trouve pas. Sans systemd, il arrête le contrôleur par le script de démarrage et en retire les liens par `update-rc.d -f batchd remove`.

# Chapitre 3

# Configuration

## 3.1 Qualificatifs du contrôleur de travaux

| Qualificatif | Défaut | Effet |
|---|---|---|
| `/ROOT=`*chemin* | `/var/spool/batch` | La racine du sous-système |
| `/JOB_LIMIT=`*n* | 16 | Travaux qui peuvent s'exécuter à la fois |
| `/QUEUE=`*liste* | `SYS$BATCH` | Les files, avec leurs limites : `(SYS$BATCH,lent:1,vif:3)` |
| `/LCORE=`*liste* | — | Confine la charge aux processeurs nommés et l'y répartit |
| `/LOG_RETAIN=`*n* | 7 | Jours pendant lesquels un journal orphelin est gardé |
| `/FOREGROUND` | — | Ne se détache pas ; pour systemd et pour le dépannage |
| `/VERBOSE` | — | Émet aussi les messages d'information |

Un qualificatif s'écrit `/NOM` ou `--NOM` ; un tiret seul n'a pas ce sens.

## 3.2 Modifier le service

```
# systemctl edit batchd
```

```
[Service]
ExecStart=
ExecStart=/usr/local/sbin/batchd /FOREGROUND /ROOT=/var/spool/batch /JOB_LIMIT=8
```

```
# systemctl daemon-reload
# systemctl restart batchd
```

La première ligne `ExecStart=`, vide, est nécessaire : sans elle la nouvelle s'ajouterait à l'ancienne au lieu de la remplacer.

## 3.3 Réglages fixés à la construction

| Réglage | Défaut |
|---|---|
| BATCH_K_SPOOLROOT | /var/spool/batch |
| BATCH_K_SHELL | /bin/sh |
| BATCH_K_MAXJOB | 512 |
| BATCH_K_NPARM | 8 |
| BATCH_K_MAXQUE | 16 |

## 3.4 Où l’utilitaire regarde

La racine vient de la variable d’environnement `BATCH_ROOT`, et à défaut de la valeur fixée à la construction. L’utilitaire et le contrôleur doivent s’accorder sur elle : le socket y est.

## 3.5 Ce qu'un travail hérite

Un travail hérite du contrôleur son environnement, son masque d'affinité — sauf `/CPU` ou `/LCORE` — et sa priorité d'ordonnancement. Il n'hérite ni du terminal, ni du répertoire de travail du contrôleur : il démarre là d'où il a été soumis.

Le sous-système ajoute à son environnement :

| Variable | Contenu |
|---|---|
| `BATCH_ENTRY` | Le numéro d'entrée |
| `BATCH_NAME` | Le nom du travail |
| `BATCH_QUEUE` | La file |
| `BATCH_RESTART` | `TRUE` si le travail a été relancé, `FALSE` sinon |
| `BATCH_RESTARTS` | Le nombre de relances |
| `P1` … `P8` | Les valeurs de `/PARAMETERS` |

Les huit `P1` … `P8` sont toujours posées ; une valeur non donnée est vide. Le sous-système ne pose rien d'autre : ni `HOME`, ni `USER`, ni `LOGNAME`, ni `SHELL`, ni `PATH`, pas même avec `/USER`.

Tout le reste vient du contrôleur, et le contrôleur a l'environnement avec lequel il a été lancé. Sous systemd, c'est celui de l'unité `batchd` : systemd lui donne `PATH` (sous Debian `/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin`), `LANG` et quelques variables à lui comme `INVOCATION_ID`, mais ni `HOME` ni `USER`. Sous le script System V, c'est celui du script. Le `PATH` d'un travail est donc le `PATH` de l'unité.

L'environnement de celui qui soumet n'est **pas** transmis : `batch submit` envoie au contrôleur le script, les qualificatifs et le répertoire courant, pas vos variables. Un `PATH` posé dans votre shell ou dans `~/.profile` n'est pas vu par le travail.

Un travail qui a besoin d'un `PATH` précis, de `HOME` ou d'une autre variable la pose lui-même, en tête du script :

```
#!/bin/sh
PATH=/opt/batch/bin:/usr/local/bin:/usr/bin:/bin
HOME=$(getent passwd "$(id -un)" | cut -d: -f6)
export PATH HOME
```

Pour changer l'environnement de tous les travaux à la fois, changez celui du contrôleur, par exemple par `Environment=` dans un complément (`systemctl edit batchd`), puis relancez-le. Les travaux déjà en cours gardent l'environnement avec lequel ils ont démarré.

# Chapitre 4

# Travailler avec les travaux par lots

## 4.1 Soumettre un travail

```
$ batch submit /opt/travaux/nuit.sh
%BATCH-S-QUEUED, job nuit queued on SYS$BATCH as entry 17
```

Le nom du travail est, par défaut, celui du fichier privé du suffixe `.sh`. Le numéro d'entrée est ce qui sert ensuite à désigner le travail.

## 4.2 Transmettre des données au travail

```
$ batch submit sauvegarde.sh '/PARAMETERS=(complète,/srv/data)'
```

Chaque valeur parvient au script deux fois : comme `P1` … `P8` et comme arguments positionnels `$1` … `$8`. Les valeurs sont remises à l'interpréteur en éléments distincts du vecteur d'arguments et jamais réunies en ligne de commande ; une valeur portant des caractères que l'interpréteur interpréterait ne peut donc rien faire exécuter.

Les parenthèses sont lues par l'interpréteur qui appelle BATCH : mettez le qualificatif entier entre apostrophes.

## 4.3 La sortie d’un travail

La sortie standard et la sortie d’erreur vont dans le journal du travail, créé par défaut dans le répertoire d’où le travail a été soumis et nommé d’après le travail et son numéro d’entrée. Si l’on ne peut y écrire, le journal va dans `<racine>/log/`.

`/LOG_FILE` nomme un autre fichier ; un tel fichier n’est jamais supprimé par le sous-système, c’est le vôtre.

Le journal d’un travail qui réussit disparaît avec son entrée, sauf si `/KEEP` a été donné.

## 4.4 Régler le moment de l’exécution

```
$ batch submit nuit.sh /AFTER=+7-
%BATCH-S-QUEUEDUNTIL, job nuit queued on SYS$BATCH as entry 4, holding until  28-AUG-2026 20:03:37
```

La forme relative est `+[jours-]hh[:mm[:ss]]`. Le champ des jours se distingue de celui des heures par le tiret qui le suit, et par rien d’autre : `+7` vaut sept heures, `+7-` sept jours. L’erreur ne se verrait qu’une fois le délai écoulé ; aussi la soumission nomme-t-elle l’heure de départ.

`/HOLD` retient sans heure : le travail attend `RELEASE`.

Ce que `SUBMIT` a réglé peut être changé sans retirer l’entrée : `SET ENTRY` modifie sur place la priorité, l’heure, la retenue, le nom et le reste, et l’entrée garde son numéro.

```
$ batch set entry 19 /PRIORITY=200 /AFTER=TOMORROW
%BATCH-S-ENTMODIFIED, entry 19 modified: priority=200, after= 9-SEP-2026 00:00:00
```

## 4.5 Afficher l’état des travaux

```
$ batch show /all

 Entry  Job name             State      Submitted            Elapsed   Status / not before
 -----  -------------------- ---------- -------------------- --------- --------------------
     1  nuit                 Holding    21-AUG-2026 20:03:37 00:00:00   28-AUG-2026 20:03:37
```

La dernière colonne dit ce qu’il est advenu du travail, et pour un travail retenu jusqu’à une heure, cette heure : connaître le moment du départ ne demande donc pas `/FULL`.

`/QUEUES` affiche les files plutôt que les travaux ; `/MANAGER`, l’état du contrôleur lui-même.

## 4.6 Supprimer un travail

```
$ batch delete 1,3,4
```

La liste s’écrit comme DCL les écrit : nombres et intervalles séparés par des virgules. Un travail qui s’exécute reçoit le signal de fin ; son entrée est traitée quand le processus a fini.

`PURGE` retire les entrées des travaux qui ont fini — le rangement dont a besoin une file qui garde ses échecs.

`/CONFIRM` fait demander avant d’agir : ce que `DELETE` et `PURGE` font ne se défait pas.

# Chapitre 5

# Dictionnaire des commandes

Une commande s’abrège à tout début non ambigu de son nom, comme un qualificatif : `SUBM` vaut `SUBMIT` et `DEL` vaut `DELETE`, tandis que `S` est refusé, `SHOW` et `SUBMIT` commençant tous deux par cette lettre.

# BATCH SUBMIT

Place un script d'interpréteur en file d'attente.

```
BATCH SUBMIT spécification-de-script [/qualificatifs]
```

| Qualificatif | Effet |
|---|---|
| `/AFTER=`*heure* | Retient le travail jusqu'à l'heure dite. Annexe B. |
| `/CPU=`*liste* | Lie le travail aux processeurs nommés. |
| `/HOLD` | Place le travail dans l'état Holding. |
| `/KEEP` | Conserve le journal après une fin heureuse. |
| `/LOG_FILE=`*fichier* | Nomme le journal. Ouvert sous l'identité du travail. |
| `/NAME=`*nom* | Nomme le travail. Un nom, non un chemin. |
| `/PARAMETERS=(`*p1,...*`)` | Jusqu'à huit valeurs. |
| `/PRIORITY=`*n* | Priorité dans la file, 1 à 255, 100 par défaut. |
| `/QUEUE=`*file* | La file. Elle doit être une de celles que le contrôleur connaît. |
| `/RESTART` | Le travail est repris si le contrôleur a été arrêté pendant son exécution. |
| `/RETAIN=`*condition* | `ALWAYS`, `ERROR` — le défaut — ou `NEVER`. |
| `/USER=`*nom* | Exécute le travail sous ce nom. Réservé à l'utilisateur privilégié. |

Une spécification de fichier et un qualificatif commencent tous deux par une barre oblique ; un argument n'est donc tenu pour un qualificatif que si le texte jusqu'au signe égal ne porte pas de barre à lui.

# BATCH SHOW

Affiche les entrées de la file.

```
BATCH SHOW [numéro-d-entrée] [/qualificatifs]
```

| Qualificatif | Effet |
|---|---|
| `/ALL` | Ajoute les entrées des travaux réussis conservées par `/RETAIN=ALWAYS`. Sans lui sont affichés les travaux en cours, en attente (Pending, Holding) et conservés après échec. |
| `/EVERYONE` | Les travaux de tous. Le privilégié n'en a que faire. |
| `/FULL` | Chaque entrée en entier. |
| `/MANAGER` | L'état du contrôleur plutôt que la file. |
| `/PAGE` | Retient l'affichage au bas de chaque écran. |
| `/QUEUE=`*file* | Restreint à une file. |
| `/QUEUES` | Les files : limite, comptes, et si elles sont suspendues. |

# BATCH DELETE

Retire des entrées, en terminant les travaux en cours.

```
BATCH DELETE liste-d-entrées [/CONFIRM]
```

# BATCH RELEASE

Libère un travail de l’état Holding.

```
BATCH RELEASE liste-d-entrées
```

Toute heure `/AFTER` qu’il portait est effacée : un travail libéré n’est pas retenu une seconde fois.

# BATCH SET ENTRY

Modifie sur place les attributs d’un travail qui est dans la file.

```
BATCH SET ENTRY liste-d-entrées [/PRIORITY=n] [/AFTER=heure | /NOAFTER] [/HOLD | /NOHOLD]
                                [/NAME=nom] [/RETAIN=mot-clé] [/KEEP | /NOKEEP]
                                [/RESTART | /NORESTART] [/CPU=liste]
```

L’entrée garde son numéro et sa place dans l’ordre ; rien n’est retiré, rien n’est soumis de nouveau. Seuls les attributs qu’un qualificatif nomme sont touchés, et la confirmation les énumère. Une commande qui n’en nomme aucun est refusée.

| État | Peut être modifié |
|---|---|
| `Pending`, `Holding` | tout |
| `Executing`, `Aborting` | `/RETAIN`, `/KEEP`, `/RESTART` seulement |
| `Retained` | rien |

Une demande qui nomme un attribut que l’état interdit est refusée en entier : une entrée n’est jamais modifiée à moitié.

`/HOLD` et `/AFTER` agissent ensemble comme à la soumission. `/NOHOLD` libère un travail retenu et efface son heure, comme `RELEASE`. Un travail retenu qui porte une heure est libéré quand l’heure vient ou quand `/NOAFTER` l’efface ; un travail retenu par `/HOLD` seul attend. `/HOLD` avec `/NOAFTER` signifie donc : retenu, sans heure. La retenue s’applique avant l’heure, de sorte que `/NOHOLD /AFTER=t` libère le travail puis le retient jusqu’à t.

`/NAME` change ce que montrent les affichages et rien d’autre : le fichier journal a été nommé à la soumission et garde son nom.

```
$ batch set entry 17 /PRIORITY=200
%BATCH-S-ENTMODIFIED, entry 17 modified: priority=200
```

# BATCH PURGE

Retire les entrées des travaux qui ont fini.

```
BATCH PURGE [/QUEUE=file] [/CONFIRM]
```

Un travail qui s’exécute n’est pas touché : PURGE range ce qui est fini, il n’arrête pas ce qui ne l’est pas.

# BATCH SYNC

Attend la fin d’un travail et rapporte comment il s’est terminé. Le code de retour porte la réponse.

```
BATCH SYNC numéro-d-entrée [/TIMEOUT=secondes]
```

```
batch sync 17 /TIMEOUT=3600 || echo "le travail nocturne s'est mal passé"
```

# BATCH QUEUE

Suspend une file, ou la laisse repartir. Réservé à l'utilisateur privilégié.

```
BATCH QUEUE nom-de-file {/PAUSE | /RESUME}
```

Une file suspendue ne démarre rien ; ses entrées ne sont pas dérangées et ses travaux en cours vont jusqu'au bout.

# BATCH ANALYZE

Examine le journal de la file et rapporte ce qui ne va pas.

```
BATCH ANALYZE [spécification-de-journal] [/DUMP]
```

Le fichier est lu directement, non par l’entremise du contrôleur ; la commande sert donc là où le contrôleur ne démarre pas. Rien n’est écrit.

# BATCHD

Le contrôleur de travaux. Ses qualificatifs sont au chapitre 3.

```
BATCHD [/qualificatifs]
```

# Chapitre 6

# Messages

## 6.1 Forme d’une valeur de condition

Toute routine du sous-système rend une valeur de condition à la manière d’OpenVMS : le numéro d’installation, le numéro du message et la sévérité. Le bit de poids faible est l’indicateur de réussite — une issue s’éprouve donc sans égard à la condition qu’elle porte.

| Sévérité | Lettre | Sens |
|---|---|---|
| Success | S | L’opération a réussi |
| Informational | I | Un renseignement ; rien n’a mal tourné |
| Warning | W | L’opération s’est faite, mais pas comme demandé |
| Error | E | L’opération n’a pas eu lieu |
| Fatal | F | L’opération n’a pas eu lieu et le programme s’arrête |

Les sévérités S et I portent le bit de poids faible ; W, E et F ne le portent pas.

## 6.2 Forme d’un message

```
%BATCH-E-NOSUCHENT, no queue entry 99
 |     | |          |
 |     | |          le texte, avec ce qui le rend particulier
 |     | la mnémonique
 |     la sévérité
 l'installation
```

Un message émis par le contrôleur porte l’horodatage et le numéro du processus ; un message de l’utilitaire, non.

## 6.3 Références des messages

Le texte anglais de chaque message est celui que produit le logiciel ; la conduite à tenir est donnée en français.

**BATCH-E-NOSUCHENT**, no queue entry *n*

L'entrée nommée n'existe pas. Elle a pu s'en aller entre-temps : une entrée dont le travail a réussi quitte la file. Vérifiez par `BATCH SHOW /ALL`.

**BATCH-E-NOSUCHQUE**, there is no queue named *nom*

La file nommée n'est pas une de celles dont le contrôleur a été informé. `BATCH SHOW /QUEUES` les énumère ; le qualificatif `/QUEUE` du contrôleur les fixe.

**BATCH-E-NOCONNECT**, job controller is not available

Le contrôleur n'est pas joignable. Voyez la section 7.1.

**BATCH-F-INUSE**, another job controller is already serving *socket*

Un contrôleur dessert déjà cette racine. Deux contrôleurs sur une même racine écriraient un même journal tour à tour, chacun ignorant la file de l'autre.

**BATCH-E-NOSCRIPT**, script *fichier raison*

Le script n'est pas utilisable : absent, illisible par le soumetteur, pas un fichier ordinaire, ou pas un chemin absolu.

**BATCH-E-SCRIPTCHG**, entry *n*: script *fichier* was replaced since it was submitted

Le fichier n'est plus celui qui a été soumis. Le script est épinglé à la soumission par périphérique, inœud, taille, date et système de fichiers ; un fichier remplacé depuis n'est pas exécuté.

**BATCH-I-SCRIPTDEV**, entry *n*: script *fichier* is on the same file system under device *maj:min*, was *maj:min*

Le disque a reçu un autre numéro de périphérique au démarrage (deux cartes trouvées dans l'autre ordre, un disque USB branché ailleurs). Le fichier et son système de fichiers sont les mêmes : le travail est exécuté.

**BATCH-E-JOBFAIL**, job *n* (*nom*) terminated with status *s*, log *fichier*

Le travail a fini sur un code non nul. Un code au-dessus de 128 est rapporté avec le signal qu'il suppose : l'interpréteur rend 128 plus le numéro du signal quand la commande qu'il exécutait a été tuée. Le journal du travail dira le reste.

**BATCH-E-JOBABORT**, job *n* (*nom*) terminated by signal *s*

Le travail lui-même a été terminé par un signal — par `DELETE`, ou par le système.

**BATCH-E-SYNCFAIL**, job *n* ended with *statut*

Le travail qu'attendait SYNC n'a pas réussi. Son journal porte ce que le script a dit.

**BATCH-E-SYNCTMO**, job *n* has not ended within *n* seconds

Le délai fixé par /TIMEOUT a expiré. Le travail est encore dans la file ; SHOW dira dans quel état.

**BATCH-S-PURGED**, *n* entries removed from the queue

`PURGE` a retiré les entrées des travaux qui avaient fini.

**BATCH-I-NOTCONF**, not confirmed, nothing was done

La question de `/CONFIRM` a reçu autre chose qu'un mot commençant par Y. Rien n'a été fait.

**BATCH-S-ENTMODIFIED**, entry *n* modified: *liste*

`SET ENTRY` a modifié l'entrée. La liste nomme les attributs touchés ; comparez-la à ce qui a été tapé.

**BATCH-E-NOTMODIFIABLE**, entry *n* is *état*, the attribute cannot be altered now

L'état du travail interdit le changement : un travail qui s'exécute n'admet que `/RETAIN`, `/KEEP` et `/RESTART`, un travail retenu après sa fin rien. S'il le faut, retirez l'entrée et soumettez de nouveau.

**BATCH-E-NOTHINGSET**, no attribute to set was given

`SET ENTRY` n'a reçu aucun qualificatif. Nommez l'attribut à modifier.

**BATCH-E-NOPRIVUSER**, uid *n* may not submit a job to run as *nom*

Seul l'utilisateur privilégié peut demander `/USER`.

**BATCH-E-NOPRIVQUE**, uid *n* may not pause or resume queue *nom*

Seul l'utilisateur privilégié peut suspendre une file ou la laisser repartir : suspendre retient le travail de tous ceux qui s'en servent. Adressez-vous à l'administrateur.

**BATCH-S-RUNASUSER**, job *n* submitted by uid *u* to run as *nom*

Trace d'un travail soumis pour s'exécuter sous un autre nom. Tout usage et tout refus sont consignés.

**BATCH-E-JRNVERSION**, journal *fichier* is incompatible: magic=*m* format=*n*

Le journal est d'un format que cette révision ne lit pas — écrit avant que les enregistrements ne se décrivent eux-mêmes. La file repart vide.

**BATCH-W-QUEORPHAN**, queue *nom* was not configured; restored paused for its entries

Le journal portait des entrées d'une file dont le contrôleur n'a pas été informé. La file est rétablie, et rétablie suspendue : les entrées sont préservées et la raison de leur immobilité se lit dans `SHOW /QUEUES`.

**BATCH-E-IVLCORE**, /LCORE=*liste* names a processor this system has not

La liste de `/LCORE` nomme un processeur inexistant. Elle est vérifiée au démarrage.

**BATCH-E-LOGSPEC**, the job log specification cannot be constructed

Le plus souvent : un nom de travail portant une barre oblique ou un renvoi au répertoire parent. Un nom doit être un nom.

# Chapitre 7

# Dépannage

Les diagnostics du contrôleur sont le premier endroit à regarder :

```
# journalctl -u batchd              # lorsqu'il tourne en service
# cat /var/spool/batch/batchd.log   # lancé à la main et détaché
```

## 7.1 Le contrôleur est injoignable

`%BATCH-E-NOCONNECT`. Trois causes, dans l'ordre de fréquence.

**Il ne tourne pas.**

```
# systemctl status batchd           # sous systemd
# systemctl start batchd
# service batchd status             # sans systemd
# service batchd start
```

**L'utilitaire et le contrôleur ne s'accordent pas sur la racine.** Le socket est sous la racine ; si `BATCH_ROOT` diffère de ce qu'a reçu le contrôleur, l'utilitaire cherche là où il n'y a rien.

```
# systemctl show batchd -p ExecStart | tr ' ' '\n' | grep ROOT
$ echo "${BATCH_ROOT}"
```

**Les droits.** Le socket est en 0666, mais le chemin qui y mène doit être traversable.

## 7.2 Le contrôleur ne démarre pas

Le terminal est averti avant que le contrôleur ne se détache :

```
%BATCH-F-CTLNOSTART, job controller did not start, root ..., log ...
```

La raison est dans le journal nommé. Les plus courantes : `%BATCH-F-INUSE` — un contrôleur dessert déjà cette racine — et un défaut de droits sur la racine.

Après `pkill batchd`, le successeur attend quelques secondes que le prédécesseur lâche son socket ; `pkill batchd && batchd` fonctionne donc.

## 7.3 Un travail ne démarre pas

Il reste Pending. Regardez, dans cet ordre :

**Les places sont prises.** `BATCH SHOW /QUEUES` donne la limite de la file et ce qui s'y exécute.

**La file est suspendue.** La même commande le dit ; `BATCH QUEUE nom /RESUME` la relance.

**Il attend une heure.** `BATCH SHOW` la montre dans sa dernière colonne.

## 7.4 Un travail échoue

`%BATCH-E-JOBFAIL`. Le journal du travail porte ce que le script a dit ; `BATCH SHOW n /FULL` porte le reste.

Un code de 126 signale que le travail n'a pu être lancé : script disparu ou remplacé, répertoire de travail inaccessible, journal impossible à créer. Un code de 127 signale que l'interpréteur n'a pas trouvé une commande ; le plus souvent c'est le `PATH` : le travail a celui du contrôleur (de son unité systemd), pas le vôtre. Posez `PATH` dans le script (voir 3.5). Un code au-dessus de 128 signale que la commande qu'exécutait l'interpréteur a été tuée.

## 7.5 La file est vide après un redémarrage

Si le journal est d'un format ancien, il est refusé et la file repart vide — `%BATCH-E-JRNVERSION`. Depuis que les enregistrements se décrivent eux-mêmes, cela ne se produit plus qu'en franchissant les révisions les plus anciennes.

`BATCH ANALYZE` lit le fichier directement et dit ce qu'il en est.

## 7.6 Le répertoire des journaux grossit

Les journaux orphelins sont balayés une fois l'heure, passé `/LOG_RETAIN` jours. Les journaux d'entrées vivantes ne le sont pas : `BATCH PURGE` retire les entrées des travaux qui ont fini, et leurs journaux s'en vont avec elles.

## 7.7 Une commande est refusée

`%CLI_RTNS-F-...` vient des services d'analyse de la ligne de commande : la forme de la commande est en cause. `%BATCH-E-...` vient du sous-système : la commande était bien formée, mais n'a pu être exécutée.

Un qualificatif s'écrit `/NOM` ou `--NOM`. Un chemin absolu n'est pas pris pour un qualificatif : la distinction se fait sur la barre oblique à l'intérieur du nom.

## 7.8 Le contrôleur répond lentement

Le journal de la file est réécrit en entier et vidé sur disque à chaque changement. Sur une file de plusieurs centaines d’entrées, une opération qui écrit le journal coûte quelques millisecondes. `BATCH PURGE` raccourcit la file et donc le journal.

## 7.9 Rassembler de quoi faire un rapport

```
# journalctl -u batchd --since '1 hour ago' > batchd.txt
$ batch show /all /full >> batchd.txt
$ batch show /manager >> batchd.txt
$ batch analyze /dump >> batchd.txt
```

# Annexe A

# Fichiers

| Chemin | Contenu |
|---|---|
| `<racine>/batch.sock` | Le socket où le contrôleur reçoit les requêtes |
| `<racine>/journal.dat` | Le journal de la file |
| `<racine>/queues.dat` | Les limites et l'état des files |
| `<racine>/batchd.log` | Le journal du contrôleur lui-même |
| `<racine>/log/` | Les journaux des travaux sans autre place |

Le journal de la file est écrit dans un fichier temporaire puis renommé par-dessus la copie précédente : on ne le voit donc jamais à demi écrit. Son enregistrement se décrit lui-même — une suite de valeurs étiquetées, non l'image d'une structure — de sorte que le journal survit à un changement de révision dans les deux sens.

## Variables d’environnement

| Variable | Lue par | Effet |
|---|---|---|
| `BATCH_ROOT` | l’utilitaire et le contrôleur | La racine du sous-système |
| `BATCH_ENTRY`, `BATCH_NAME`, `BATCH_QUEUE` | le travail | Son identité |
| `BATCH_RESTART`, `BATCH_RESTARTS` | le travail | S’il a été relancé, et combien de fois |
| `P1` … `P8` | le travail | Les valeurs de `/PARAMETERS` |

Ce sont les seules variables que le sous-système pose pour un travail. Toutes les autres, `PATH` compris, viennent de l’environnement du contrôleur (voir 3.5).

# Annexe B

# Formes de spécification du temps

## B.1 Temps relatif

`+[jours-]hh[:mm[:ss]]`

Les champs omis valent zéro. Le champ des jours se distingue de celui des heures par le tiret qui le suit, et par rien d'autre :

| Écriture | Sens |
|---|---|
| `+7` | sept heures |
| `+7-` | sept jours |
| `+7-12:00` | sept jours et douze heures |
| `+:30` | trente minutes |
| `+::45` | quarante-cinq secondes |

## B.2 Temps absolu

```
jj-MMM-aaaa[:hh:mm:ss]
```

```
/AFTER=29-JUL-2026
/AFTER=29-JUL-2026:03:00
```

L’heure est lue à l’horloge de qui l’écrit, et affichée à l’horloge de qui la lit. Le sous-système tient l’UTC au-dedans ; la conversion prend le décalage en vigueur à l’instant nommé, non à l’instant présent, de sorte qu’une heure de l’autre côté d’un changement de saison est juste.

## B.3 Mots-clés

| Mot | Sens |
|---|---|
| `NOW` | Maintenant |
| `TODAY` | Le minuit par lequel a commencé le jour courant |
| `TOMORROW` | Le minuit par lequel commence le jour suivant |